



Fake it 'till you make it

**Zarr-like Access of Existing NetCDF4
Datasets**

Lucas Sterzinger

*Mentors: Chelle Gentemann (Farallon Institute), Julia Kent (NCAR), Kevin
Paul (NCAR)*

July 28, 2021



The Problem

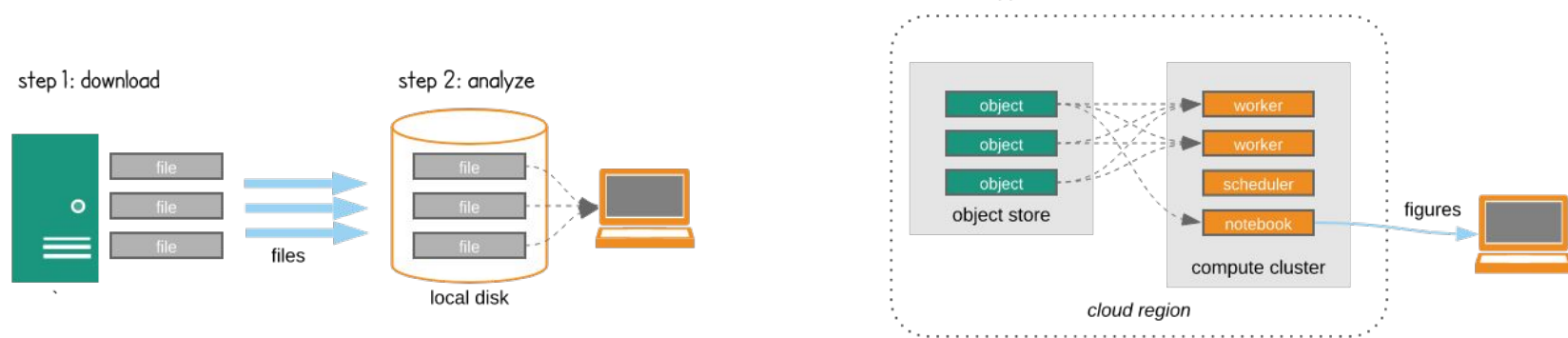
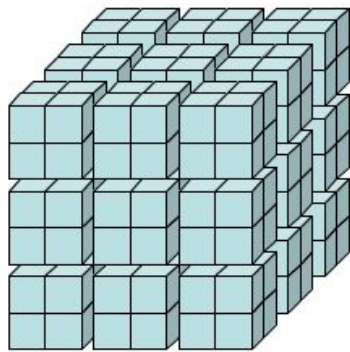


Image Source - Pangeo/Ryan Abernathy

- Peta(Exa?)bytes of geophysical data now available in the cloud
- Much of the data are in NetCDF4/HDF5 format
 - Not efficient for cloud computing

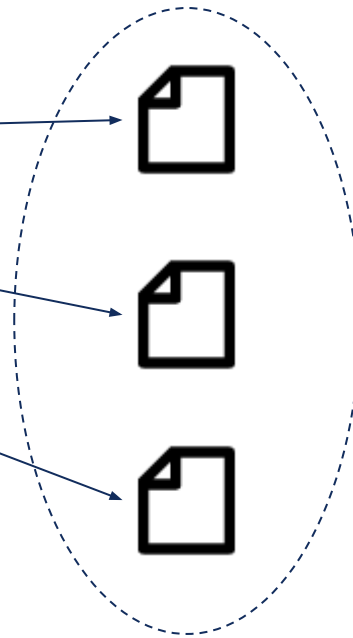
NetCDF vs Zarr

NetCDF



Single File Object

Zarr



Multiple File Objects
(One per chunk)



Plaintext
Metadata
File

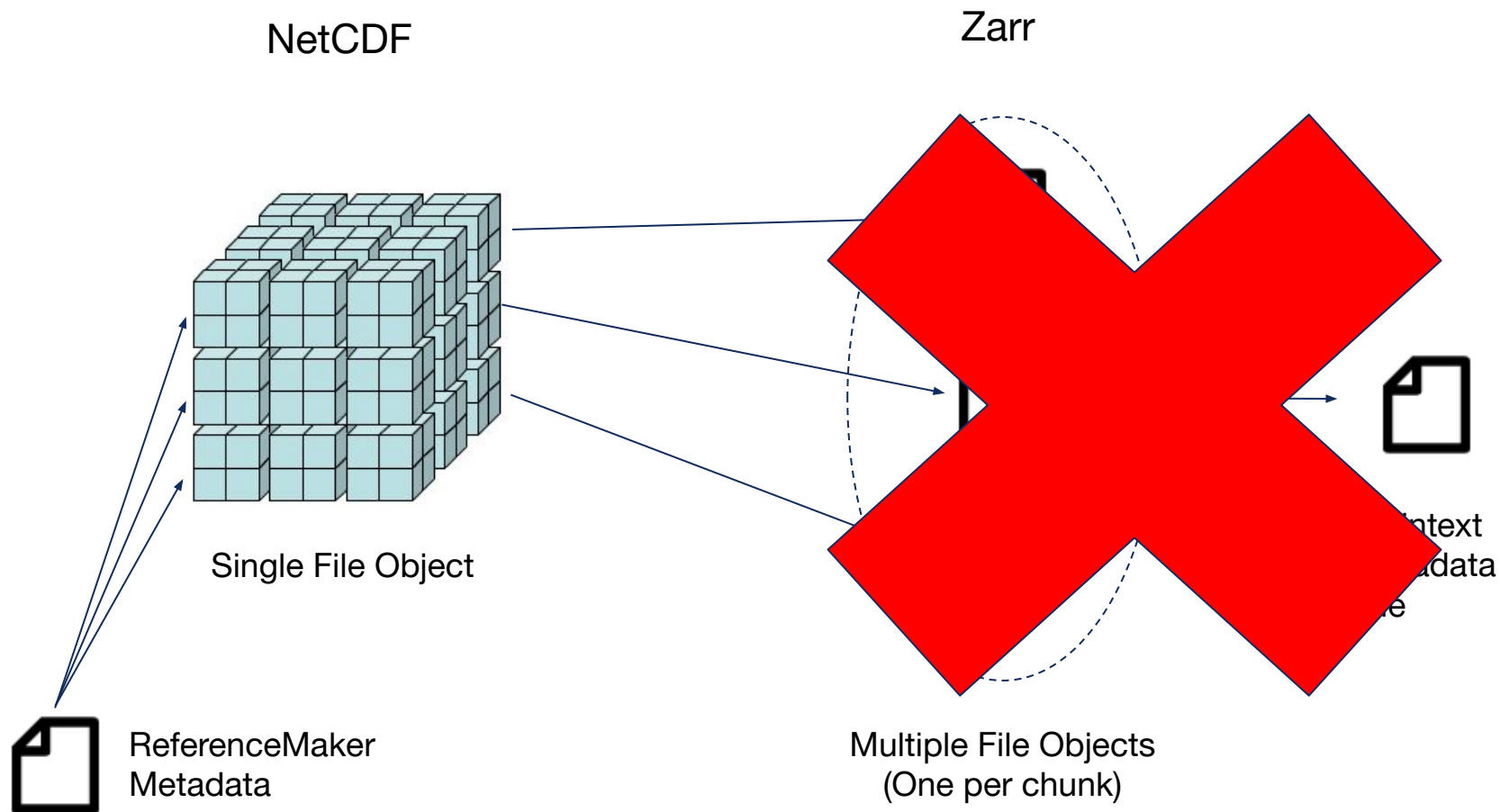
How does it work?

ReferenceMaker:

- Part of Intake group's **fsspec** project
- Access existing NetCDF/HDF files *as if they are Zarr*
- Zarr-like Metadata files (few MB each)
 - Remote file address (Azure Blob, AWS S3, etc)
 - Data variable and chunking info
 - Can be read as a Zarr file
- Extracts byte-ranges for individual variables/chunks
 - Instead of individual chunk files in Zarr



ReferenceMaker



Creating JSONS

`@dask.delayed` ←

Easily distributed with Dask!

```
def gen_json(url):  
    so = dict(  
        mode="rb", anon=True, default_fill_cache=False, default_cache_type="none"  
    )
```

```
    outpath = './jsons/' + url.split('/')[-1].split('.nc')[0] + '.json'
```

```
    with fsspec.open(url, **so, account_name='goesuwest') as inf:  
        h5chunks = SingleHdf5ToZarr(inf, u, inline_threshold=300)
```

Read remote file and
process chunks

```
    with open("./reference.json", 'wb') as outf:  
        outf.write(ujson.dumps(h5chunks.translate()).encode())
```

Write metadata to file

Reference JSON Example

The screenshot shows a JSON file viewer with a tab labeled "OR_ABI-L2-MCMIPF-M6_G". The JSON structure is as follows:

```
root:
  version: 1
  u: "az://noaa-goes16/ABI-L2-MCMIPF/2020/002/00/OR_ABI-L2-MCMIPF-M6_G16_s20200020000216_e20200020009524_c20200020010031.nc"
  .zattrs: {
    "Convent": "GOES-R Series Advanced Baseline Imager",
    "date_created": "2020-01-02T00:10:03.1Z",
    "id": "087244ef-58ee-4c60-a351-2d8b63086250",
    "institution": "DOC/NOAA/NESDIS > U.S. Department of Commerce, National Oceanic and Atmospheric Administration, National Environmental Satellite, Data, and Information Services",
    "instrument_ID": "FM1",
    "instrument_type": "GOES R Series Advanced Baseline Imager",
    "iso_series_metadata_id": "8c9e8150-3692-11e3-aa6e-0800200c9a66",
    "keywords": "ATMOSPHERE > ATMOSPHERIC RADIATION > REFLECTANCE, SPECTRAL/ENGINEERING > VISIBLE WAVELENGTHS > REFLECTANCE, SPECTRAL/ENGINEERING > INFRARED WAVELENGTHS > BRIGHTNESS TEMPERATURE",
    "keywords_vocabulary": "NASA Global Change Master Directory (GCMD) Earth Science Keywords, Version 7.0.0.0.0",
    "license": "Unclassified data. Access is restricted to approved users only.",
    "naming_authority": "gov.nesdis.noaa",
    "orbital_slot": "GOES-East",
    "platform_ID": "G16",
    "processing_level": "National Aeronautics and Space Administration (NASA) L2",
    "production_data_source": "n/a",
    "production_environment": "OE",
    "production_site": "NSOF",
    "project": "GOES",
    "scene_id": "Full Disk",
    "spatial_resolution": "2km at nadir",
    "standard_name_vocabulary": "CF Standard Name Table (v25 - 20 July 2014)"
  }
```

A green box highlights the value of the "u" field: "az://noaa-goes16/ABI-L2-MCMIPF/2020/002/00/OR_ABI-L2-MCMIPF-M6_G16_s20200020000216_e20200020009524_c20200020010031.nc".

A blue box highlights the text: "File URL (Azure Storage for this example)".

Reference JSON Example

```
ministration (NASA) L2 , production_data_source : n/a , production_environment : OE ,  
"production_site": "NSOF", "project": "GOES", "scene_id": "Full Disk", "spatial_resolution":  
n": "2km at nadir", "status": "Final", "table": "Table (v35, 20 July 2016)", "summary": "Multipl  
6)", "summary": "Multipl  
ducts are digital maps o  
and IR bands.", "time_co  
20-01-02T00:00:21.6Z", "timeline_id": "ABI Mode 6", "title": "ABI L2 Cloud and Moisture I  
magery" }
```

Variable (CMI_C01) chunk info,
compression, shape, etc

```
CMI_C01/.zarray: "{ \"chunks\": [ 226, 226 ], \"compressor\": { \"id\": \"zlib\", \"level\": 1 }, \"dt  
ype\": \"<i2\", \"fill_value\": -1, \"filters\": null, \"order\": \"C\", \"shape\": [ 5424, 5424 ], \"z  
arr format\": 2 }"
```

```
CMI_C01/.zattrs: "{ \"_ARRAY_DIMENSIONS\": [ \"y\", \"x\" ], \"_FillValue\": -1, \"_Netcdf4Dimid\":  
0, \"_Unsigned\": \"true\", \"add_offset\": 0.0, \"ancillary_variables\": \"DQF_C01\", \"cell_method  
s\": \"t: point area: sum (interval: 0.000028 rad)\", \"coordinates\": \"band_id_C01 band_wavel  
length_C01 t y x\", \"downsampling_method\": \"average\", \"grid_mapping\": \"goes_imager_projecti  
on\", \"long_name\": \"ABI Cloud and Moisture Imagery reflectance factor\", \"resolution\": \"y:  
0.000056 rad x: 0.000056 rad\", \"scale_factor\": 0.00031746001332066953, \"sensor_band_bit_d  
epth\": 10, \"standard_name\": \"toa_lambertian_equivalent_albedo_multiplied_by_cosine_solar_  
zenith_angle\", \"units\": \"1\", \"valid_range\": [ 0, 4095 ] }"
```

```
▼ CMI_C01/0.4: [ ] 3 items  
0: "{u}"  
1: 253643  
2: 468
```

Single Chunk Byte Range

GOES Workflow



Data stored in same region as compute



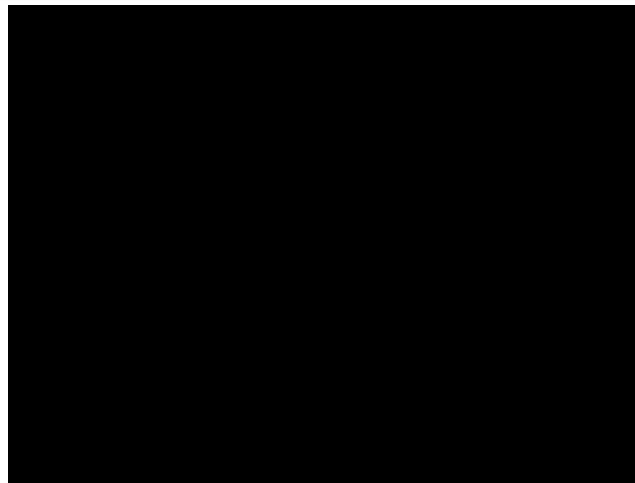
4 CPU, 32 GB Memory,
8 Dask Workers

- Created a test workflow on Microsoft Planetary Computer
- Analysis of 24 hours of GOES data
- Test access to data stored in Azure in 3 formats:
 - NetCDF4
 - NetCDF4 + ReferenceMaker
 - Zarr

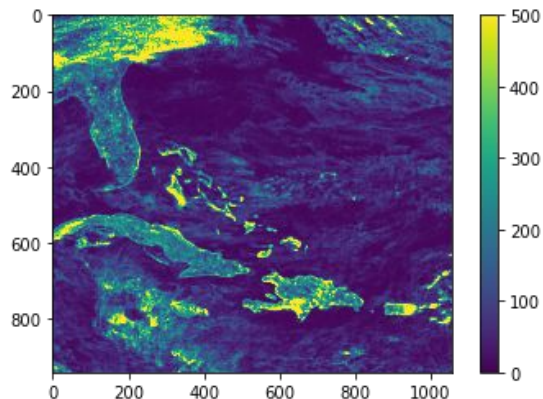
GOES Workflow



Data stored in same region as compute



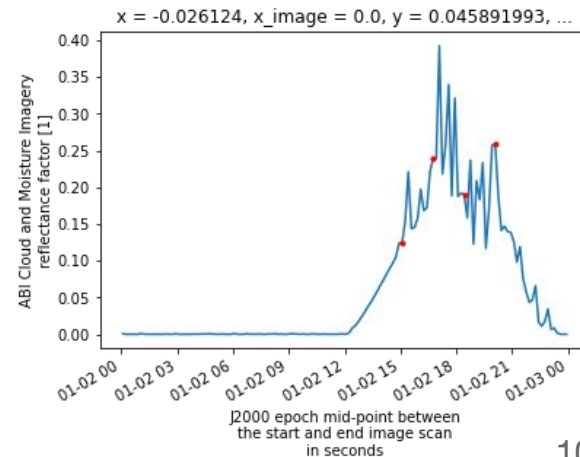
4 CPU, 32 GB Memory,
8 Dask Workers



True Color 2020-01-02 0005 UTC



Veggie Color 2020-01-02 0005 UTC



GOES Processing Times

For 24 hours of GOES-16 “ABI-L2-MCMIPF” product data

<u>Format</u>	<u>Preprocess Time</u>	<u>Data Open Time</u>	<u>Workflow Time</u>	<u>Extra Storage</u>
Remote NetCDF4	0 min	10 minutes	40 min	0 GB
Zarr	1 h 38 min	30 seconds	4 min	52 GB
ReferenceMaker	1 h 25 min	35 seconds	5 min 30 s	<u>416 MB</u>

- ReferenceMaker wins in data-open time and storage cost
- NetCDF has no upfront preprocess time cost
 - But very slow open and workflow times
- Zarr comes at cost of high storage costs (full data duplication)

This is a big deal!

- ReferenceMaker allows users to harness the cloud-optimization of Zarr without needing to convert any data
- Allows current institutions to cheaply make existing cloud-hosted data more optimized
- Reference files can be also created and hosted by third parties
- Automatable
 - e.g. Pangeo Forge

PANGEO

Microsoft Planetary
Computer



SpatioTemporal
Asset Catalog



You can help!

The more scientists and developers talk to each other, the better this ecosystem becomes.

- Join the Pangeo community
- Talk to project maintainers about your workflow needs
- Open issues/bug reports on GitHub
- Submit pull requests, even if your code isn't 100% ready

Acknowledgements

Thank you to the ReferenceMaker team!

- Special thanks to Martin Durant (Anaconda) and Rich Signell (USGS) for their help learning and contributing to this project
- <https://github.com/intake/fsspec-reference-maker>

Thank you to Kevin Paul (NCAR), Julia Kent (NCAR), and Chelle Gentemann (Farallon Institute) for their amazing mentorship through the SIParCS program

Thanks to the SIParCS team for working so hard to make a fully-online internship experience so successful

Thanks to Microsoft for providing credits for Planetary Computer

Please feel free to reach out with questions
lsterzinger@ucdavis.edu or twitter [@lucassterzinger](https://twitter.com/lucassterzinger)



Link to site with interactive code examples, tutorials, resources, and contact info

<https://lucassterzinger.com/2021-siparcs-poster/>